Ranked Latent Memory with Depth-Profiled FFN Reallocation

Aditya Shankar **Sparsh Bhartia** **Udith Krishna M Nair** **Adithya Kishor**
{aditya.shankar, sparsh.bhartia,
udithkrishna.nair, adithya.kishor}@research.iiit.ac.in

Abstract

Engram (Cheng et al., 2026) and Lngram (Zheng et al., 2026) attach a large lookup table to a Transformer so that common local patterns can be read from memory instead of rebuilt through several layers of computation. Both show this frees up depth: useful information appears earlier in the network than in a matched baseline. Neither then makes the backbone smaller to take advantage of it. The one experiment in that direction, Lngram's 23-layer variant, deletes a whole Transformer block; it improves the average score but drops sharply on MMLU and PIQA, which suggests it also removed computation the memory cannot replace. Our project asks where the backbone can actually afford to lose capacity. We shrink feed-forward width across the range of layers where the memory appears to be doing the work, keep the early and late layers at full width, and move the saved parameters into the table, which can be served from system RAM rather than GPU memory. We decide where to shrink by measuring which feed-forward layers the memory made redundant, rather than assuming it.

1 Introduction

A Transformer has no built-in way to look something up. When it needs a fact or a fixed phrase, it reconstructs it through attention and feed-forward computation. Feed-forward layers behave like key-value memories (Geva et al., 2021), and much of the early and middle stack goes into assembling entities from their parts. Lngram gives a clear example: in a 24-layer model, "Diana, Princess of Wales" is built up over roughly six layers, starting as a vague geographic concept and only settling on the specific person at layer 6.

Two recent papers replace that reconstruction with a real lookup. **Engram** (Cheng et al., 2026) builds the lookup key by hashing the token IDs of the last few tokens. Because the key depends only on the input text, the model knows which rows it will need before computing anything, so they can be fetched from system RAM in advance; the authors report under 3% slowdown with a 100B-parameter table held in host memory. **Lngram** (Zheng et al., 2026) builds the key from the model's own hidden state instead, projecting it and taking the sign of each dimension to get bits, which are grouped into short codes that index the table. The keys become learnable and stop depending on the tokenizer. We use Lngram as our starting point.

Both report the same effect on the backbone. Lngram's CKA analysis shows its layers resembling *deeper* layers of the baseline, a gain of two to three layers that is strongest in the middle of the stack. The obvious follow-up, which neither paper really pursues, is whether the backbone can then be made smaller.

2 Motivation

2.1 Existing work removes capacity bluntly

Lngram tries this once. They remove one full Transformer block from their 24-layer model, and the resulting 23-layer version still beats the 24-layer baseline on average (0.5207 against 0.5146) with fewer parameters. That is real evidence that memory can stand in for part of the backbone.

The per-task numbers are more informative than the average. The 23-layer model gains on HellaSwag, SciQ and WinoGrande but loses on MMLU (0.2619 down to 0.2300, close to random guessing) and PIQA (0.6926 down to 0.6740). The authors conclude that Lngram replaces a subset of local pattern matching rather than general Transformer depth.

We think the problem is *where* and *how* the capacity was removed. Deleting a block takes out attention and the feed-forward network together, at one depth, all at once. If the memory is replacing one specific kind of work, assembling local phrases, and that work happens in a particular range of lay-

ers, then what should be removed is feed-forward *width* across *that range*, leaving attention alone. There is enough in these papers to guess the range, and a cheap way to check the guess.

2.2 The lookup returns a fixed vector

In Lngram all retrieved rows are concatenated and passed through one fixed projection. Which rows are retrieved depends on the input, but how they are combined does not, and no individual row is scored for relevance. UltraMemV2 (Huang et al., 2025a) showed a clear gain from replacing such fixed vectors with tiny one-unit-wide feed-forward networks, at the same compute and the same bytes fetched.

2.3 The table does not have to sit on the GPU

Engram keeps a 100B-parameter table in system RAM at under 3% slowdown, and Lngram measures only 1.4% extra GPU memory for its own module, with nothing that grows as the sequence gets longer. That points at a way to run larger models on a small GPU: keep the computation on the GPU and let the CPU serve the lookup table out of ordinary system RAM, which is cheap and plentiful compared to VRAM. Neither paper is written for this setting, and neither reports how much GPU memory a memory-augmented model actually occupies.

It also changes what a sensible parameter split looks like. Both papers divide parameters between memory and MoE experts and settle on roughly 20 to 25% in memory. We use a dense model, so there are no experts to trade against and feed-forward width is the only place the parameters can come from, and as far as we can tell nobody has measured that trade.

3 Contributions

1. **Targeted feed-forward reallocation (main contribution).** Rather than deleting a layer, we shrink feed-forward width across the range where the memory adds effective depth, and check whether this avoids the MMLU and PIQA losses that deletion caused. We compare against Lngram at the same total parameter count, so the question is how to allocate a memory-augmented model, not whether memory helps.

2. **A measurement that tells us where to cut.** We compare per-layer feed-forward importance between a model with memory and one without, which shows where the memory took over. A cheap version of the same measurement, run beforehand on an existing model, tells us where to cut instead of guessing.

3. **Reporting GPU memory alongside parameter count.** Since the table can live in system RAM, we also report resident GPU memory and single-token decode latency, which for local deployment are often the real constraints.

4. **Optional extensions.** Small per-slot networks in place of fixed vectors, a delayed injection point, and a scored readout (Section 5.4).

4 Related Work

Engram and Lngram are described in Section 1 and are our baselines. One detail matters later: thresholding a hidden state into bits is not differentiable, so Lngram trains its key projection with a surrogate gradient that compares what would have been retrieved under nearby codes.

Two other papers contribute pieces we reuse. UltraMemV2 (Huang et al., 2025a) contributes the small-per-slot-network design and the finding that how many entries are active per token matters more than how many exist in total. UltraMem (Huang et al., 2025b) introduced computing a lookup at one layer and injecting it at a later one, so the lookup overlaps with computation.

MoE (Dai et al., 2024) is the compute-side counterpart to conditional memory and is the backbone in both Engram's and Lngram's experiments, which is why their allocation results do not carry over to a dense model.

5 Proposed Methodology

5.1 Backbone

A standard modern dense Transformer: SwiGLU feed-forward blocks, grouped-query attention, rotary position embeddings and RMSNorm. We deliberately use the same hidden size, depth and head count as Lngram's models so that their architectural findings apply to us directly.

This lets us borrow their *architectural* settings, such as which n-gram orders to use, how many bits per code, and where to insert the module. Their optimizer settings are tuned for a 2B MoE model on 35B tokens, so learning rate, batch size and warmup have to be retuned for ours; we treat their values as starting points.

	S	M
d_{model} / layers	1024 / 24	1280 / 28
d_{ff} (baseline)	2816	3456
Attention / KV heads	16 / 2	20 / 4
Feed-forward params	208M	372M
Backbone total	298M	528M
Memory table	100M	175M
Total	**≈400M**	**≈700M**

Table 1: Model configs. planned runs use S; M is a stretch target

5.2 Memory module

We use Lngram's module as published: inserted at layers 2 and 12 as a side branch before attention, using 2-gram and 3-gram orders, 4 bits per code, and the cheaper approximate version of their surrogate gradient. We keep two training rules they found necessary, namely a larger learning rate with no weight decay on table parameters, and starting the small convolution at the end of the branch at zero so the branch does not disturb the backbone early on. We also follow UltraMemV2's initialization rule, which matches the branch's output variance to the feed-forward network's.

This style of addressing should suit small models. Capacity grows by adding codes and subtables rather than by enlarging a hash space, and every token updates one entry per code per order, so entries get trained fairly evenly. A hashed table has no such guarantee: X-GRAM (Chen et al., 2026) found that most entries in a large hashed table receive almost no updates, and reports Engram getting *worse* as its table grows at sub-1B scale. That is the regime we are working in, and it is the main reason we build on Lngram rather than Engram.

5.3 Where to shrink the feed-forward layers

Three findings in Lngram point at the same region. Their CKA analysis puts the effective-depth gain at two to three layers, strongest in the middle. Their LogitLens analysis shows layer 1 briefly doing worse than baseline, then pulling ahead from the middle onward. Their worked example has entity assembly finishing around layer 6 of 24. For a 24-layer model with the memory at layers 2 and 12, our starting hypothesis is to keep layers 0 to 2 at full width, since they produce the hidden state the lookup key is computed from; reduce width across layers 3 to 16, the span between and just past the insertion points; and keep layers 17 to 23 at full width for final composition and output.

This is a redistribution rather than a reduction: whatever width we remove goes into the table, so every setting has the same total parameter count. Halving d_{ff} across 14 layers moves about 61M parameters, which sets the scale. We sweep how much to cut and whether to cut evenly across the span or more deeply near the insertion points, and we leave the insertion layers themselves at full width, since the memory branch reads their hidden state to decide how much of the retrieved content to admit. If training turns out to be unstable with the reduced widths, we can warm up briefly without the memory branch and switch it on afterwards.

5.4 Optional extensions

Small networks per slot. Store two vectors per table entry instead of one. Reading an entry then means taking the dot product of the hidden state with the first vector, passing it through a nonlinearity, and scaling the second vector by the result. This is the smallest possible feed-forward network, one hidden unit wide, so the entry does a little computation on the hidden state instead of just being added to it, at the same bytes fetched per token.

Delayed injection. Compute the key at layer ℓ but inject the result at layer $\ell + \Delta$ (Huang et al., 2025b), giving the lookup time to finish while the GPU works on other layers. This matters because Lngram computes and injects at the same layer, and their own measurements show single-token decoding running 6.7% slower than baseline as a result, even though prefill is 1.17 times faster. Setting $\Delta = 0$ recovers standard Lngram, so the sweep tells us what hiding the latency costs in quality.

Scored readout. Fetch more candidate rows than needed, score each against the current hidden state, and keep only the best. Every design in this line fetches a fixed set and uses all of it. Scoring should help most when the table is small and unrelated patterns are forced to share entries, which is our situation.

A longer-term direction. Learned keys and hashed keys each give up something. Hashed keys cannot generalize across different wordings of the same thing, but because they are known in advance they can be prefetched, and individual facts can be written into known rows afterwards. Learned keys generalize but lose both properties. A table split into a hash-keyed region for editable facts and a latent-keyed region for general patterns, sharing

one readout, would get both. This is beyond what we can attempt this semester, but it is the direction we would take the work next.

6 Dataset and Training Setup

We pretrain on ClimbMix (Diao et al., 2025), a 400B-token mixture built for efficient pretraining that outperforms other corpora at an equal token budget, and which the authors validated at 350M and 1B scale. Since we are training much smaller models than Engram and Lngram did, our numbers will not be directly comparable to theirs either way, so we prefer the corpus that gets more out of a limited budget. FineWeb-Edu (Penedo et al., 2024), which Lngram used, is our fallback.

The budget is roughly Chinchilla-optimal at about 20 tokens per parameter, so around 8B tokens for configuration S. Lngram's scaling results suggest staying near this rather than training longer, since their advantage over the baseline shrinks as the token budget grows and the dense backbone starts learning those local patterns itself.

If we run the optional frozen-backbone finetuning experiments, we need a domain the base model is visibly weak on; we would use medical text, training on PubMed abstracts and evaluating on MedQA or PubMedQA. However, Lngram found that every domain-adaptation method hurt general benchmarks, so the approach may be to train the dense part generally and rely on the lookup table for domain facts.

Compute budget. We have four independent machines on the Ada cluster, one with 4x RTX 3090 and three with 4x RTX 2080 Ti, so four jobs can train in parallel. Each core run is about 265M non-embedding parameters over 8B tokens, roughly 1.3×10^{19} FLOPs. At a realistic 30% utilisation that is about 180 GPU-hours on the 3090 machine and 230 on a 2080 Ti machine, so two to three days each; the 2080 Ti cards lack bfloat16, so we train in fp16 there. The four core runs come to about 860 GPU-hours and three days in parallel, and the sweeps, at a quarter of the token budget, add about 340 more. With evaluation and a 20% buffer this is roughly 1,500 GPU-hours and about a week of training, well inside the schedule. External compute not needed, but that or turing would help a lot if access to turing is possible. Contingency if our compute is insufficient: Just drop ablation runs.

7 Evaluation Plan

Metrics. Validation loss and perplexity, plus zero- and few-shot accuracy on the benchmark set both Engram and Lngram use, run with `lm-eval` (Biderman et al., 2024): HellaSwag (Zellers et al., 2019), MMLU (Hendrycks et al., 2021), SciQ (Welbl et al., 2017), WinoGrande (Sakaguchi et al., 2021) and PIQA (Bisk et al., 2020). MMLU and PIQA get the most attention, since those are the two tasks Lngram's layer-deleted model lost on, and MMLU is also the most reasoning-heavy task in the set, so it is where we would first see a problem if shrinking feed-forward layers hurts multi-step reasoning. Differences here tend to be under a point, so close results will be checked by resampling the evaluation examples. We also measure resident GPU memory, prefill throughput, and decode latency at batch size one.

Core comparisons. Three runs carry the argument and are the priority: Lngram with feed-forward layers untouched at the same total parameter count, which is the baseline we claim to improve on; uniform layer deletion at the same parameter count, reproducing Lngram's 23-layer experiment in our dense setting, which is the alternative we are arguing against; and our targeted width reduction.

Additional work as time allows. We do not expect to run all of these, and will pick among them once the core comparison is in. A per-layer importance profile on the trained models, to check whether the memory absorbed work in the layers we shrank rather than across the stack; a cheaper version of this runs first, on an existing model, to decide where to shrink. Beyond that: a plain dense Transformer at the same parameter count and at the same validation loss; Engram at the same parameter count, which would tell us whether learned keys are what make this work at small scale; sweeps over how much to shrink and how much of the budget goes to the table; and the LogitLens and CKA analyses Lngram used.

8 Tentative Timeline

Weeks 1 to 2: implement and verify the Lngram module, and measure layer importance to decide where to shrink. Weeks 3 to 4: dense and Lngram reference runs. Weeks 5 to 6: uniform layer deletion and our targeted reduction. Weeks 7 to 8: analysis, sweeps and writing up. Remaining time goes to the optional extensions.

References

Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar Van Der Wal. 2024. Lessons from the trenches on reproducible evaluation of language models. *arXiv preprint arXiv:2405.14782*.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Yilong Chen, Yanxi Xie, Zitian Gao, He Xin, Yihao Xiao, Jason Klein Liu, Haoming Luo, Yifan Luo, Zhengmao Ye, Tingwen Liu, Xin Zhao, Ran Tao, and Bryan Dai. 2026. Beyond n-gram: Data-aware x-gram extraction for efficient embedding parameter scaling. *arXiv preprint arXiv:2604.21724*.

Xin Cheng, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, Kezhao Huang, Xingkai Yu, Zhewen Hao, Yukun Li, Han Zhang, Huishuai Zhang, Dongyan Zhao, and Wenfeng Liang. 2026. Conditional memory via scalable lookup: A new axis of sparsity for large language models. *arXiv preprint arXiv:2601.07372*.

Damai Dai, Chengqi Deng, Chenggang Zhao, Ruoxi Xu, Huaqian Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Yuxuan Wu, et al. 2024. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*.

Shizhe Diao, Yu Yang, Yonggan Fu, Xin Dong, Dan Su, Markus Kliegl, Zijia Chen, Peter Belcak, Yoshi Suhara, Hongxu Yin, Mostofa Patwary, Yingyan Lin, Jan Kautz, and Pavlo Molchanov. 2025. Climb: Clustering-based iterative data mixture bootstrapping for language model pre-training. *arXiv preprint arXiv:2504.13161*.

Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. 2021. Transformer feed-forward layers are key-value memories. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5484–5495.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring massive multitask language understanding. In *International Conference on Learning Representations*.

Zihao Huang, Yu Bao, Qiyang Min, Siyan Chen, Ran Guo, Hongzhi Huang, Defa Zhu, Yutao Zeng, Banggu Wu, Xun Zhou, and Siyuan Qiao. 2025a. Ultramemv2: Memory networks scaling to 120b parameters with superior long-context learning. *arXiv preprint arXiv:2508.18756*.

Zihao Huang, Qiyang Min, Hongzhi Huang, Defa Zhu, Yutao Zeng, Ran Guo, and Xun Zhou. 2025b. Ultrasparse memory network. In *International Conference on Learning Representations*.

Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. 2024. The fineweb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106.

Johannes Welbl, Nelson F Liu, and Matt Gardner. 2017. Crowdsourcing multiple choice science questions. In *Proceedings of the 3rd Workshop on Noisy Usergenerated Text*.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*.

Yunao Zheng, Guoyang Xia, Xiaojie Wang, and Lei Ren. 2026. Lngram: N-gram conditional memory in latent space. *arXiv preprint arXiv:2605.24869*.